

# 00a - Önceliklendirilmiş Eylem Planı

Olusturulma: 14 Nisan 2026 19:49 Kaynak: docs/audit Format: Tekil rapor

## 00a - Önceliklendirilmiş Eylem Planı

Tarih: 2026-04-13 Temel rapor: docs/audit/00-overall-audit.md Kapsam: P0/P1/P2 audit maddelerini eyleme dönüştürme. Bu dosya plan dokümanıdır; kod değişikliği içermez.

### Öncelik özeti

- P0: Doğrulanmış P0 yok. İlk genel taramada doğrudan veri kaybı, yetkisiz toplu erişim veya uygulamanın tamamen açılmaması seviyesinde kanıt bulunmadı.
- P1: Test kapısının kırık olması, yazdırma/encoding karmaşıklığı, monolitik kritik dosyalar, paketleme/node\_modules bağımlılık döngüsü ve frontend bundle büyümesi.
- P2: Frontend test/lint/typecheck eksikleri, API client ve App seviyesinde sorumluluk yığılması, migration/runtime config disiplin borcu, script karmaşası.

### Ürün / İş Kuralı

#### P1 - Sipariş ve ödeme iş kuralları büyük route dosyalarında yoğun

- Sorun özeti: Sipariş oluşturma, item lifecycle, paket sipariş, mutfak job üretimi, ödeme kapatma ve masa temizleme gibi iş kritik kurallar büyük route dosyalarında toplanmış.
- Kullanıcıya/işletmeye etkisi: Hatalı masa kapatma, eksik mutfak fişi, yanlış ödeme durumu veya müşteri istatistiği gibi doğrudan operasyonel problemler çıkabilir.
- Teknik kök neden: server/routes/orders.js 627 satır, server/routes/payments.js 389 satır; route, transaction, domain kuralı ve yan etki aynı katmanda.
- Önerilen çözüm: Önce davranışı kapsayan integration/unit testleri güçlendir; sonra sipariş/ödeme domain servisleri çıkar. Route dosyaları yalnız HTTP doğrulama ve servis çağırısı yapmalı.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta-yüksek.

#### P2 - Runtime config kaynakları iş davranışını da etkiliyor

- Sorun özeti: .env, pos-config.json, Electron env üretimi, bridge env ve DB settings aynı davranış alanlarını etkiliyor.
- Kullanıcıya/işletmeye etkisi: Aynı işletmede farklı makinelerde farklı yazıcı/port/token/encoding davranışı oluşabilir; destek maliyeti artar.
- Teknik kök neden: Config tek doğruluk kaynağına bağlanmamış; bridge ayarları electron/main.cjs, server/routes/bridge.js, store-bridge/printers/renderers.js boyunca yayılmış.

- Önerilen çözüm: Config precedence dokümanı ve validasyon katmanı ekle. Yazıcı profilini tek nesne olarak normalize et.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

## UX/UI

### P1 - Büyük frontend bundle düşük donanımlı POS cihazlarını zorlar

- Sorun özeti: Client build başarılı ama ana chunk 948.07 kB minified; XLSX chunk 429.03 kB.
- Kullanıcıya/işletmeye etkisi: Electron veya eski kasalarda açılış süresi ve bellek tüketimi artar; yoğun servis saatlerinde yavaşlık algısı oluşur.
- Teknik kök neden: Route bazlı code splitting yok; ağır ekranlar ve kütüphaneler ilk yüklemeye yakın konumda.
- Önerilen çözüm: React lazy/Suspense ile route bazlı bölme; `xlsx`, reports/charts ve printer settings gibi ağır alanları dynamic import ile ayır.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta.

### P2 - App seviyesinde modal, role, route ve navigation state yığılmış

- Sorun özeti: `client/src/App.jsx` route tanımı, auth role kararları, sidebar, ödeme modal state'i ve navigation fallback'lerini aynı bileşende taşıyor.
- Kullanıcıya/işletmeye etkisi: Ödeme sonrası yanlış ekrana dönme, sidebar görünürlük hataları veya role bazlı erişim davranışında kırılabilirlik oluşabilir.
- Teknik kök neden: Top-level component çok fazla UI orchestration sorumluluğu üstleniyor.
- Önerilen çözüm: Protected route, payment modal orchestration ve route config parçalara ayrılmalı; önce küçük, davranış değiştirmeyen extraction yapılmalı.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

## Frontend Mimarisi

### P1 - OrderScreen monolitik ve profesyonel bakım standardının altında

- Sorun özeti: `client/src/components/orders/OrderScreen.jsx` 1428 satır; ekran bileşeni iş kuralı, API orchestration, state ve UI detaylarını aynı yerde taşıyor.
- Kullanıcıya/işletmeye etkisi: Sipariş ekranında yapılacak küçük değişiklikler ürün ekleme, toplam hesaplama, ödeme tetikleme veya mutfak akışını bozabilir.
- Teknik kök neden: Hook, utility ve alt component sınırları yeterince ayrılmamış.
- Önerilen çözüm: Önce saf hesaplama ve format fonksiyonlarını utility'ye çıkar; sonra API orchestration hook; en son UI alt bileşenleri.
- Risk seviyesi: P1.
- Uygulama zorluğu: Yüksek.

## P1 - PrinterDetailPage çok büyük ve entegrasyon UI'ı kırılğan

- Sorun özeti: `client/src/components/settings/PrinterDetailPage.jsx` 972 satır; yazıcı konfigürasyon UI'ı tek dosyada yoğun.
- Kullanıcıya/işletmeye etkisi: Yanlış encoding, yanlış cihaz seçimi veya kaydedilen ayarların bridge ile uyumsuz olması yazdırmayı bozabilir.
- Teknik kök neden: Form state, discovery seçimi, printer option normalizasyonu ve UI aynı dosyada.
- Önerilen çözüm: Printer form modelini utility/hook olarak ayır; encoding/device option alanlarını küçük komponentlere böl.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta-yüksek.

## P2 - API client tek sınıfta tüm domainleri topluyor

- Sorun özeti: `client/src/services/api.js` auth, masa, ürün, sipariş, ödeme, müşteri, caller ID, rapor, stok, waiter call ve print uçlarını tek sınıfta topluyor.
- Kullanıcıya/işletmeye etkisi: Yeni endpoint eklemeleri kolay görünür ama uzun vadede yanlış domain çağrısı, copy-paste hata ve test zorluğu üretir.
- Teknik kök neden: Domain bazlı client modülleri yok.
- Önerilen çözüm: Davranışı koruyarak domain modüllerine böl; geriye uyum için mevcut `api` facade bir süre korunabilir.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

## Backend / API

### P1 - Test kapısı kırık olduğu için API değişiklikleri güvenle doğrulanamıyor

- Sorun özeti: `npm test --prefix server` `vitest` bulunamadığı için çalışmıyor.
- Kullanıcıya/işletmeye etkisi: Ödeme, sipariş, yazdırma gibi para ve operasyon akışlarında regresyon riski artar.
- Teknik kök neden: Server devDependency tanımlı ama yerel `server/node_modules/.bin/vitest` mevcut değil veya kurulum bozuk.
- Önerilen çözüm: Server bağımlılıklarını kur; test komutunu root ve CI için tek güvenilir kapı yap; testlerin çalıştığını audit dosyasında kaydet.
- Risk seviyesi: P1.
- Uygulama zorluğu: Düşük.

### P1 - Yazdırma job üretimi ve mock/bridge davranışı çok katmana yayılmış

- Sorun özeti: Backend print job üretimi, mock processing, bridge claim ve renderer farklı dosyalarda doğru ayrılmış olsa da profil/encoding kararları dağınık.
- Kullanıcıya/işletmeye etkisi: Fiş çıkmaması, yanlış yazıcıya gitmesi veya Türkçe karakter bozulması işletmenin canlı kullanımını doğrudan etkiler.
- Teknik kök neden: `server/services/printJobs.js`, `server/routes/bridge.js`, `store-bridge/jobs/poller.js`, `store-bridge/printers/renderers.js` arasında net profil kontratı zayıf.

- Önerilen çözüm: Print payload schema ve printer profile contract dokümanite edilmeli; renderer snapshot testleri artırılmalı.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta.

## Veritabanı

### P2 - Migration dosyası büyümüş ve sürdürülebilir sürümleme zayıf

- Sorun özeti: `server/migrations/run.js` 673 satır; schema ve veri düzeltme mantığı aynı dosyada büyüyor.
- Kullanıcıya/işletmeye etkisi: Sahadaki eski veritabanlarından yükseltmede beklenmeyen kolon/indeks/veri uyumsuzluğu riski artar.
- Teknik kök neden: Migrationlar küçük, sıralı ve geri izlenebilir dosyalara ayrılmamış.
- Önerilen çözüm: Mevcut idempotent yaklaşımı koruyarak yeni migrationları numaralı dosyalara ayır; migration audit tablosu veya schema version ekle.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta-yüksek.

## Entegrasyonlar

### P1 - Türkçe karakter/yazıcı kod sayfası cihaz bağımlı ve kırılğan

- Sorun özeti: PC857, Windows-1254, ESC t ve Phoenix clone komutları aynı akışta env/DB/printer option ile çözülüyor.
- Kullanıcıya/işletmeye etkisi: Müşteri fişinde veya mutfak fişinde `ÇĞİÖŞÜ çğıöşü` bozulur; ürün adları anlaşılmaz hale gelir, marka güveni düşer.
- Teknik kök neden: Encoding kararı tek bir profil sözleşmesine bağlanmamış; `win1254` default, `escT=32`, `skipPhoenixCmd` gibi varsayımlar cihaz modeline bağlı.
- Önerilen çözüm: Yazıcı profilleri oluştur: `generic-pc857`, `jp80h-win1254`, `ascii-safe` gibi. Profil test fişi ve byte snapshot ekle.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta.

### P2 - Caller ID ve StoreBridge operasyonel süreçleri aynı desktop lifecycle içinde yoğun

- Sorun özeti: Electron main process server, bridge ve caller ID helper lifecycle'larını birlikte yönetiyor.
- Kullanıcıya/işletmeye etkisi: Bridge restart veya caller ID hatası masaüstü uygulamanın genel stabilitesini etkileyebilir.
- Teknik kök neden: `electron/main.cjs` 860 satır ve process yönetimi tek dosyada.
- Önerilen çözüm: Process manager yardımcı modülleri çıkar; log ve health state ayrıştır.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

## Desktop / Paketleme

### P1 - `file:..` dependency ve paketleme çıktıları tarama/paketleme döngüsü yaratıyor

- Sorun özeti: Alt paketler root projeyi `file:..` dependency olarak çekiyor; `node_modules/restoran-pos` altında recursive repo kopyaları/tarama hataları oluşuyor.
- Kullanıcıya/işletmeye etkisi: Build/paketleme yavaşlar, Windows path/symlink/antivirüs sorunları artar; destek verilebilirlik düşer.
- Teknik kök neden: Workspace yerine dosya bağımlılığıyla root paket alt paketlerin içine bağlanmış.
- Önerilen çözüm: npm workspaces tasarla veya root dependency ihtiyacını kaldır; dist ve node\_modules tarama kapsamından sistematik dışlanmalı.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta-yüksek.

### P1 - Electron main process çok fazla sorumluluk taşıyor

- Sorun özeti: Config okuma, JWT üretimi, DB taşıma, backend spawn, bridge spawn, caller ID helper ve BrowserWindow aynı dosyada.
- Kullanıcıya/işletmeye etkisi: Desktop açılış, kapanış, port çakışması, bridge restart gibi sorunların kök nedeni zor bulunur.
- Teknik kök neden: `electron/main.cjs` 860 satır; process lifecycle modüler değil.
- Önerilen çözüm: `config`, `serverProcess`, `bridgeProcess`, `callerIdProcess`, `sqliteMigration` modüllerine ayırıştır.
- Risk seviyesi: P1.
- Uygulama zorluğu: Yüksek.

### P2 - Root script yüzeyi operasyonel olarak karışık

- Sorun özeti: Root `package.json` içinde dev, prod, Electron, native rebuild, dist, caller ID ve Windows batch komutları yoğun.
- Kullanıcıya/işletmeye etkisi: Yanlış komut çalıştırma ve eksik build alma riski artar.
- Teknik kök neden: Script ayrımı görev bazlı gruplanmamış.
- Önerilen çözüm: README'deki kullanıcı komutları ile developer/release komutlarını ayrı dokümente et; script isimlerini netleştir.
- Risk seviyesi: P2.
- Uygulama zorluğu: Düşük-orta.

## Test / QA

### P1 - Server test altyapısı çalışmıyor

- Sorun özeti: Test dosyaları var ama komut çalışmıyor.
- Kullanıcıya/işletmeye etkisi: Kritik akışlarda sessiz regresyon riski.
- Teknik kök neden: Bağımlılık kurulumu veya workspace yapısı bozuk.

- Önerilen çözüm: `npm install --prefix server`; `npm test --prefix server`; sonucu CI veya lokal gate olarak sabitle.
- Risk seviyesi: P1.
- Uygulama zorluğu: Düşük.

## P1 - Yazıcı encoding test kapsamı yetersiz

- Sorun özeti: Encoding test dosyası var ama audit bulgusu, fiziksel cihaz profili ve ESC/POS komut snapshotlarının büyütülmesi gerektiğini gösteriyor.
- Kullanıcıya/işletmeye etkisi: Türkçe karakter problemi sahada tekrar eder.
- Teknik kök neden: Byte encode testi tek başına yeterli değil; ESC t, Phoenix skip, template line wrapping birlikte doğrulanmalı.
- Önerilen çözüm: Printer preview/encoding testlerine profil bazlı snapshot ekle.
- Risk seviyesi: P1.
- Uygulama zorluğu: Düşük-orta.

## P2 - Frontend test, lint ve typecheck kapıları yok

- Sorun özeti: Client scriptleri yalnız `dev`, `build`, `preview`; root/server/client scriptlerinde lint/typecheck yok.
- Kullanıcıya/işletmeye etkisi: UI regresyonları ve basit kalite hataları build'e kadar yakalanmaz.
- Teknik kök neden: QA araçları script yüzeyine eklenmemiş.
- Önerilen çözüm: Önce davranış değiştirmeyen `lint` veya en azından `build` gate dokümanı; sonra Vitest/RTL veya Playwright smoke testleri.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

# Güvenlik / Operasyon

## P1 - Production JWT secret kontrolü olumlu, ama operasyonel test kapısı eksik

- Sorun özeti: Production fallback secret engelleniyor; ancak test komutu kırık olduğu için güvenlik middleware değişiklikleri düzenli doğrulanamıyor.
- Kullanıcıya/işletmeye etkisi: Auth veya business scope regresyonu fark edilmeden sahaya gidebilir.
- Teknik kök neden: Güvenlik kontrolleri var, QA kapısı güvenilir değil.
- Önerilen çözüm: Auth middleware ve integration testleri CI/lokal gate içinde çalışır hale getir.
- Risk seviyesi: P1.
- Uygulama zorluğu: Düşük.

## P2 - Build artifact ve node\_modules kaynak taramasını kirlletiyor

- Sorun özeti: `dist-electron` ve çoklu `node_modules` repo içinde mevcut; audit/tarama araçları bunları dolaşırken hata ve zaman aşımı üretebiliyor.
- Kullanıcıya/işletmeye etkisi: Geliştirici verimliliği düşer; yanlış dosyada analiz/değişiklik riski artar.
- Teknik kök neden: Kaynak ve çıktı ayrımı yerel workspace'te net değil; tarama komutları ignore disiplini olmadan çalışıyor.

- Önerilen çözüm: Geliştirici dokümanına kaynak tarama scope'u ekle; workspace/package stratejisini düzelt.
- Risk seviyesi: P2.
- Uygulama zorluğu: Düşük-orta.

## Önceliklendirilmiş uygulama sırası

### Hemen düzeltilmesi gerekenler

1. Server test altyapısını çalışır hale getir.
2. Yazıcı encoding/ESC-POS test kapsamını genişlet.
3. `file:..` dependency ve node\_modules recursive tarama sorununu en azından dokümante edip tarama/build kapsamından izole et.
4. Büyük frontend bundle için düşük riskli route lazy loading planını başlat.

### Bu hafta ele alınacaklar

1. `store-bridge/printers/renderers.js` içinden encoding/profile yardımcılarını çıkarmaya başla.
2. `OrderScreen.jsx` için saf utility ve hook extraction planını uygula.
3. `PrinterDetailPage.jsx` için form modelini UI'dan ayır.
4. Electron process lifecycle'ı için modül sınırlarını çıkar ve küçük refactorlara başla.
5. Migration stratejisini numaralı/idempotent migration dosyalarına taşıma planı hazırla.

### Sonraya bırakılacaklar

1. Tam npm workspaces dönüşümü.
2. Büyük route/service ayrıştırmaları.
3. Playwright tabanlı uçtan uca POS smoke testleri.
4. Electron process manager kapsamlı ayrıştırması.
5. Full printer profile registry ve cihaz model kataloğu.

## En kritik 5 sorun

1. Server test komutunun çalışmaması.
2. Türkçe karakter/yazıcı encoding zincirinin cihaz bağımlı ve dağınık olması.
3. `OrderScreen.jsx` ve yazdırma renderer'ının monolitik yapısı.
4. `file:..` dependency ile recursive node\_modules/tarama/paketleme karmaşası.
5. Frontend bundle'ın düşük donanımlı POS cihazları için büyümesi.

## En güvenli 5 hızlı kazanım

1. Server bağımlılıklarını kurup `npm test --prefix server` kapısını çalışır hale getirmek.
2. Yazıcı encoding testlerine Türkçe karakter ve TL sembolü snapshotları eklemek.
3. Audit altında derin konu raporlarını oluşturmak: printing, payment integrity, frontend state, packaging.
4. Client build uyarısını görünür yapmak ve lazy-load planını dokümante etmek.

5. Kaynak tarama komutlarında `node_modules`, `dist-electron`, `release` kapsam dışı disiplinini belgelemek.

## En riskli ama gerekli 5 refactor alanı

1. `client/src/components/orders/OrderScreen.jsx` parçalama.
2. `store-bridge/printers/renderers.js` encoding/layout/template ayrıştırması.
3. `electron/main.cjs` process lifecycle modüllerine bölme.
4. `server/routes/orders.js` ve `server/routes/payments.js` domain servislerine ayrıştırma.
5. npm workspace/package dependency yapısını yeniden tasarlama.